Memory Management in C++

Theory, Edge Cases & Code Examples

- ✓ Automatic Lifetime Management
- ✓ Manual Lifetime Management
- ✓ Direct Comparison
- ✓ Resource Acquisition Is Initialization
- ✓ Memory Management Evolution in C++

→ *Swipe to learn* →

Memory Management in C++

Theory

📌 1. Stack Memory - Automatic Lifetime Management (1/2)

Definition:
- **Automatic storage duration** - managed entirely by the compiler
- Follows **LIFO (Last In First Out)** principle
- Memory allocated during function call, freed on function return
- Part of the thread's **call stack**

Characteristics:

Property	Details
Allocation Speed	Extremely fast (single pointer increment)
Size Limit	Small: typically 1-8 MB (platform-dependent)
Lifetime	Scope-based (automatic cleanup at `}`)
Access Pattern	Sequential, highly cache-friendly
Fragmentation	None - linear allocation/deallocation
Thread Safety	Thread-local (no synchronization needed)

What Lives on the Stack:
- Local variables declared in functions
- Function parameters
- Return addresses and stack frames
- Small temporary objects
- Built-in types (`int`, `double`, `char`, etc.)

Memory Management in C++

Theory

📌 1. Stack Memory - Automatic Lifetime Management (2/2)

Example:

```
void stackDemo() {
    int x = 42;           // Stack: allocated here
    double pi = 3.14;     // Stack: allocated here
    char ch = 'A';        // Stack: allocated here

    {
        int y = 100;      // Stack: allocated in inner scope
    }                     // y destroyed here (automatic)

    // x, pi, ch still alive
}  // All stack variables destroyed here (automatic)
```

Memory Layout:

```
High Address
│
├─ int y = 100        (inner scope)
├─ char ch = 'A'
├─ double pi = 3.14
├─ int x = 42
├─ Return address
│
Low Address (Stack grows downward)
```

Performance:

- **Allocation:** ~1-2 CPU cycles (just increment stack pointer)
- **Deallocation:** ~1-2 CPU cycles (just decrement stack pointer)
- **Cache behavior:** Excellent - sequential access, high locality

When to Use Stack:

- Small objects (< 1 KB as a guideline)
- Short-lived data (within function scope)
- Fixed-size objects known at compile time
- Performance-critical code paths

Memory Management in C++

Theory

📌 2. Heap Memory - Manual Lifetime Management (1/3)

Definition:
- **Dynamic storage duration** - programmer controls lifetime
- Also called **free store** or **dynamic memory**
- Memory persists until explicitly freed
- Managed by the **system allocator** (malloc/free, new/delete)

Characteristics:

Property	Details
Allocation Speed	Slower (involves allocator algorithms)
Size Limit	Large: limited only by available system memory (GBs)
Lifetime	Manual (programmer-controlled)
Access Pattern	Random, scattered across memory
Fragmentation	Can fragment over time (external fragmentation)
Thread Safety	Requires synchronization (global allocator)

What Lives on the Heap:
- Objects created with `new` or `malloc`
- Objects whose size is unknown at compile time
- Objects that must outlive their creation scope
- Large data structures (arrays, buffers, images)
- Dynamically-sized containers (`std::vector` internal storage)

Memory Management in C++

Theory

📌 2. Heap Memory - Manual Lifetime Management (2/3)

Example:

```
void heapDemo() {
    // Heap allocation - manual lifetime
    int* ptr = new int(42);           // Allocate on heap
    int* arr = new int[1000];         // Large array on heap

    // Memory persists even after function returns
    // (until explicitly deleted)

    delete ptr;         // Must manually free single object
    delete[] arr;       // Must manually free array
}

// Example: Object outliving scope
int* createNumber() {
    int* num = new int(100);  // Heap allocation
    return num;               // ✅ Valid - heap memory persists
}  // Function returns, but heap object still alive
```

Memory Layout:

```
Heap (grows upward from low address)
│
├─ [allocated block] ← new int[1000]
├─ [free space]
├─ [allocated block] ← new int(42)
├─ [free space]
├─ [allocated block] ← earlier allocation
│
(Scattered, non-contiguous)
```

📌 2. Heap Memory - Manual Lifetime Management (3/3)

Performance:

- **Allocation:** ~100-1000+ CPU cycles (search free list, update metadata)
- **Deallocation:** ~100-1000+ CPU cycles (coalesce free blocks, update metadata)
- **Cache behavior:** Poor - random access, low locality

When to Use Heap:

- Large objects (> 1 KB)
- Objects with dynamic/unknown size at compile time
- Objects that outlive their creation scope
- Shared data across multiple scopes
- Polymorphic objects (via pointers to base class)

Memory Management in C++

Theory

📌 3. Stack vs Heap - Direct Comparison

Code Comparison:

```
// STACK ALLOCATION
void stackExample() {
    int numbers[100];  // ✅ Fast: instant allocation
    numbers[0] = 42;
}  // ✅ Instant deallocation

// HEAP ALLOCATION
void heapExample() {
    int* numbers = new int[100];  // ⏱️ Slower: allocator overhead
    numbers[0] = 42;
    delete[] numbers;             // ⏱️ Must remember to free
}

// HYBRID APPROACH (Modern C++)
void modernExample() {
    std::vector<int> numbers(100);  // ✅ Heap storage, RAII management
    numbers[0] = 42;
}  // ✅ Automatic cleanup (RAII)
```

Decision Matrix:

```
Object Size:
├─ < 1 KB           → Stack
├─ 1 KB - 100 KB    → Heap (with RAII)
└─ > 100 KB         → Heap (with RAII)

Lifetime:
├─ Within function  → Stack
├─ Across functions → Heap
└─ Unknown duration → Heap

Size Known:
├─ Compile-time     → Stack (or std::array)
└─ Runtime          → Heap (std::vector, smart pointers)
```

Memory Management in C++

Theory

📌 4. RAII - Resource Acquisition Is Initialization (1/4)

Core Principle:

Tie resource lifetime to object lifetime

The RAII Pattern:

1. **Acquire resources in constructor** (allocation, opening files, acquiring locks)
2. **Release resources in destructor** (deallocation, closing files, releasing locks)
3. **Compiler guarantees destructor runs** when object goes out of scope

Why RAII is Fundamental:

```
// ❌ MANUAL MANAGEMENT - DANGEROUS
void manualWay() {
    int* data = new int[1000];

    doSomeWork();           // What if this throws?

    if (errorCondition()) {
        return;             // ❌ LEAK! delete never called
    }

    moreWork();             // What if this throws?

    delete[] data;          // ❌ May never execute
}

// ✅ RAII - SAFE
void raiiWay() {
    std::vector<int> data(1000);  // Constructor allocates

    doSomeWork();           // ✅ Safe: destructor will run

    if (errorCondition()) {
        return;             // ✅ Safe: destructor runs
    }

    moreWork();             // ✅ Safe: destructor will run

}  // ✅ Destructor ALWAYS runs (automatic cleanup)
```

Memory Management in C++

Theory

📌 4. RAII - Resource Acquisition Is Initialization (2/4)

RAII Guarantees:

- ✅ **Deterministic cleanup** - resources freed at scope exit
- ✅ **Exception-safe** - destructors run during stack unwinding
- ✅ **No manual cleanup** - compiler handles it
- ✅ **Prevents leaks** - impossible to forget cleanup

RAII in Standard Library:

Type	Resource Managed	Constructor	Destructor
`std::unique_ptr<T>`	Heap memory (exclusive ownership)	Allocates/takes ownership	Deletes object
`std::shared_ptr<T>`	Heap memory (shared ownership)	Allocates + ref count	Decrements ref, deletes if 0
`std::vector<T>`	Dynamic array	Allocates heap storage	Deletes all elements
`std::string`	Character data	Allocates buffer	Frees buffer
`std::fstream`	File handle	Opens file	Closes file
`std::lock_guard`	Mutex lock	Acquires lock	Releases lock

Memory Management in C++

Theory

📌 4. RAII - Resource Acquisition Is Initialization (3/4)

Example - RAII with Smart Pointers:

```
#include <memory>

class Sensor {
public:
    Sensor(const std::string& name) {
        std::cout << "Sensor " << name << " initialized\n";
    }
    ~Sensor() {
        std::cout << "Sensor destroyed\n";
    }
};

void testRAII() {
    // Old way - manual management ❌
    Sensor* s1 = new Sensor("lidar");
    // ... must remember to delete ...
    delete s1;  // Easy to forget!

    // Modern way - RAII ✅
    auto s2 = std::make_unique<Sensor>("camera");
    // ... no manual cleanup needed ...
}  // s2 automatically destroyed here
```

📌 4. RAII - Resource Acquisition Is Initialization (4/4)

Exception Safety Through RAII:

```
void processData() {
    // All RAII objects
    std::vector<int> buffer(10000);
    auto file = std::make_unique<std::fstream>("data.txt");
    std::string result;

    // Even if exception thrown here...
    riskyOperation();

    // ...all destructors run during stack unwinding:
    // 1. result.~string()
    // 2. file.~unique_ptr() → closes file
    // 3. buffer.~vector()   → frees memory
}
```

Memory Management in C++

Theory

📌 5. Memory Management Evolution in C++ (1/2)

C++98/03 - Manual Era:

```
void oldWay() {
    int* arr = new int[100];
    // ... manual tracking ...
    delete[] arr;  // Easy to forget, no exception safety
}
```

- ❌ Manual new/delete everywhere
- ❌ Memory leaks common
- ❌ Exception-unsafe code
- ❌ Complex ownership tracking

C++11 - RAII Revolution:

```
void modernWay() {
    auto arr = std::make_unique<int[]>(100);
    // Automatic cleanup, exception-safe
}
```

- ✅ Smart pointers (`unique_ptr`, `shared_ptr`)
- ✅ Move semantics (efficient transfers)
- ✅ `make_unique`, `make_shared` factories
- ✅ Exception-safe by default

C++14+ - Best Practices:

```
void bestPractice() {
    std::vector<int> arr(100);  // Prefer containers
    // Zero manual memory management
}
```

📌 5. Memory Management Evolution in C++ (2/2)

- ✅ Prefer containers over raw arrays
- ✅ **Rule of Zero** - no custom destructors
- ✅ Smart pointers only when necessary
- ✅ Value semantics by default

Memory Management in C++

Theory

📌 6. Why Memory Management Matters (1/2)

Common Bugs Without Proper Management:

Bug Type	Example	Consequence	Detection
Memory Leak	Forgetting `delete`	Gradual memory exhaustion	Valgrind, ASan
Dangling Pointer	Using pointer after `delete`	Undefined behavior, crashes	ASan, nullptr checks
Double Delete	`delete` same pointer twice	Heap corruption, crash	Smart pointers
Buffer Overflow	`arr[1000]` on `arr[100]`	Memory corruption, security	Bounds checking, ASan
Use After Free	Reading freed memory	Unpredictable behavior	ASan, sanitizers
Stack Overflow	Large local arrays	Program crash	Heap allocation

Production Impact:

```
// Real-world scenario: Autonomous vehicle perception
void perceptionLoop() {
    while (true) {
        int* lidarData = new int[100000];  // ❌ Leak!
        processLidar(lidarData);
        // Missing delete[]
    }
    // After 10 minutes: Out of memory crash
    // Vehicle loses perception → CRITICAL FAILURE
}

// RAII solution
void perceptionLoopSafe() {
    while (true) {
        std::vector<int> lidarData(100000);  // ✅ RAII
        processLidar(lidarData.data());
    }  // Automatic cleanup every iteration
    // Runs reliably for days/weeks
}
```

Memory Management in C++

Theory

📌 6. Why Memory Management Matters (2/2)

Security Implications:

- **Memory leaks** → Denial of service attacks
- **Buffer overflows** → Code injection exploits
- **Use-after-free** → Remote code execution
- **Dangling pointers** → Information disclosure

Modern C++ Solution:

```
Manual Memory Management
        ↓
Use RAII Everywhere
        ↓
Prefer Containers (vector, string)
        ↓
Use Smart Pointers When Needed
        ↓
Rule of Zero (no custom destructors)
        ↓
Safe, Fast, Maintainable Code
```

Memory Management in C++

Theory

📌 7. Best Practices Summary

✅ **DO:**
- Use `std::vector` instead of raw arrays
- Use `std::unique_ptr` for exclusive ownership
- Use `std::shared_ptr` for shared ownership
- Prefer stack allocation when possible
- Use `std::make_unique` and `std::make_shared`
- Follow Rule of Zero (let compiler generate special members)

❌ **DON'T:**
- Use raw `new` / `delete` (unless absolutely necessary)
- Mix `malloc` / `free` with `new` / `delete`
- Return pointers to local variables
- Forget to `delete` allocated memory
- Use mismatched operators (`new` / `delete[]`)

Quick Decision Tree:

```
Need dynamic memory?
    │
    ├─ No  → Use stack allocation (local variables)
    │
    └─ Yes → What ownership model?
             │
             ├─ Single owner → std::unique_ptr
             │
             ├─ Shared owners → std::shared_ptr
             │
             └─ Collection → std::vector/std::array
```

Memory Management in C++

Edge Cases

📌 1. Returning Address of Local Stack Variable

Returning a pointer or reference to a local stack variable creates **undefined behavior** because the variable's memory is reclaimed when the function returns. The pointer then refers to invalid memory that may be overwritten by subsequent function calls.

```
int* dangerous() {
    int x = 42;
    return &x;  // ❌ UB: x is destroyed after return
}

void caller() {
    int* ptr = dangerous();
    std::cout << *ptr;  // ❌ Accessing invalid memory
}
```

This code may appear to work in simple test cases because the stack memory hasn't been overwritten yet, making the bug particularly insidious. Compilers may warn about this pattern, but warnings are easily missed or ignored.

Memory Management in C++

Edge Cases

📌 2. Mismatched Array Allocation and Deallocation

Using `delete` instead of `delete[]` for array allocations causes **undefined behavior**. The scalar `delete` operator only destructs the first element and may corrupt the heap allocator's internal structures.

```
int* arr = new int[100];
delete arr;  // ❌ Should be delete[]

class Widget {
    ~Widget() { std::cout << "destroyed\n"; }
};

Widget* widgets = new Widget[5];
delete widgets;  // ❌ Only first destructor runs, remaining 4 leak
```

For primitive types like `int`, this may seem benign, but for classes with destructors, it causes resource leaks. The heap corruption can manifest as crashes in unrelated code, making debugging extremely difficult.

Memory Management in C++

Edge Cases

📌 3. Dangling Pointers After Deletion

Accessing memory through a pointer after `delete` is **undefined behavior**. The memory may be immediately reused, unmapped, or marked as poisoned by debugging tools. The pointer becomes a **dangling pointer**.

```
int* p = new int(42);
delete p;
std::cout << *p;  // ❌ UB: p is now dangling

p = nullptr;  // ✅ Good practice: prevents accidental use
```

Setting pointers to `nullptr` after deletion is defensive programming that converts undefined behavior into deterministic crashes (null pointer dereference), which are much easier to debug.

Memory Management in C++

Edge Cases

📌 4. Mixing Allocation and Deallocation Mechanisms

C++ provides two distinct memory management systems: C-style (`malloc` / `free`) and C++-style (`new` / `delete`). Mixing them causes **undefined behavior** because they use different allocators with incompatible internal bookkeeping.

```
int* p1 = (int*)malloc(sizeof(int));
delete p1;  // ❌ UB: malloc must use free

int* p2 = new int(10);
free(p2);  // ❌ UB: new must use delete

int* arr = new int[10];
free(arr);  // ❌ UB: even worse mixing
```

Additionally, `new` calls constructors while `malloc` does not, and `delete` calls destructors while `free` does not, making them fundamentally incompatible for non-trivial types.

Memory Management in C++

Edge Cases

📌 5. Double Delete and Shared Ownership

Deleting the same memory twice causes **undefined behavior** and typically results in heap corruption or immediate crashes. This commonly occurs when multiple raw pointers share ownership without coordination.

```
int* p = new int(100);
int* q = p;  // Both point to same memory

delete p;
delete q;  // ❌ Double free: UB and likely crash
```

Modern C++ solves this with ownership semantics: `std::unique_ptr` for exclusive ownership and `std::shared_ptr` for reference-counted shared ownership, eliminating manual coordination.

Memory Management in C++

Edge Cases

📌 6. Exception Safety and Memory Leaks

When exceptions occur before `delete` statements, allocated memory is leaked. This is one of the primary motivations for RAII in C++.

```
void unsafe() {
    int* data = new int[1000];
    riskyOperation();  // ❌ If throws, data is leaked
    delete[] data;     // Never reached
}

void safe() {
    std::vector<int> data(1000);  // ✅ RAII: auto cleanup
    riskyOperation();  // Even if throws, vector destructor runs
}
```

The RAII pattern guarantees that destructors run during stack unwinding when exceptions are thrown, ensuring resource cleanup regardless of execution path.

Memory Management in C++

Edge Cases

📌 7. Stack Overflow with Large Objects

The stack has limited size (typically 1-8 MB). Allocating large objects on the stack can exceed this limit, causing **stack overflow** errors that may manifest as segmentation faults or silent corruption.

```
struct Massive {
    int data[1000000];  // ~4 MB
};

void stackOverflow() {
    Massive m;  // ❌ May exceed stack limit
}

void heapAlloc() {
    auto m = std::make_unique<Massive>();  // ✅ Uses heap
}
```

The stack overflow may not occur immediately if previous stack usage was minimal, making the bug environment-dependent and difficult to reproduce consistently.

Memory Management in C++

Code Examples

📌 1. Stack vs Heap Memory Allocation

Stack variables have automatic storage duration managed by the compiler through stack pointer manipulation. Heap allocations invoke the system allocator (`operator new`), which manages free store memory and requires explicit deallocation.

```
#include <iostream>

void stackExample() {
    int local = 42;  // ✅ Stack: automatic lifetime
    std::cout << "Stack: " << local << "\n";
}  // local is automatically destroyed here

void heapExample() {
    int* dynamic = new int(42);  // Heap: manual lifetime
    std::cout << "Heap: " << *dynamic << "\n";
    delete dynamic;  // ✅ Must manually free
}
```

Memory Management in C++

Code Examples

📌 2. Correct Array Allocation and Deallocation

The `delete[]` operator performs three tasks: invokes destructors for all elements, deallocates the memory block, and updates internal allocator bookkeeping. Using scalar `delete` only performs the first destructor call, creating inconsistencies.

```
void arrayManagement() {
    // ✅ Correct: matching allocation/deallocation
    int* arr1 = new int[10];
    delete[] arr1;

    // ❌ Incorrect: mismatched operators
    int* arr2 = new int[10];
    delete arr2;  // UB: should be delete[]

    // ✅ Modern approach: no manual management
    std::vector<int> arr3(10);
    // Automatic cleanup via destructor
}
```

Memory Management in C++

Code Examples

📌 3. Memory Leak from Pointer Reassignment

When reassigning a raw pointer, the original memory address is lost without deallocation, creating a memory leak. Smart pointers automatically delete managed objects when replaced or destroyed.

```
void leakExample() {
    int* ptr = new int(100);  // First allocation
    ptr = new int(200);       // ❌ Leak: lost reference to first allocation
    delete ptr;               // Only frees second allocation
}

void fixedExample() {
    std::unique_ptr<int> ptr = std::make_unique<int>(100);
    ptr = std::make_unique<int>(200);  // ✅ First automatically deleted
    // Second automatically deleted at scope exit
}
```

Memory Management in C++

Code Examples

📌 4. RAII with std::unique_ptr

The unique_ptr guarantees that the managed object's destructor runs when the smart pointer goes out of scope, regardless of how the scope is exited (normal return, exception, or early return).

```
#include <memory>

class Resource {
public:
    Resource() { std::cout << "Resource acquired\n"; }
    ~Resource() { std::cout << "Resource released\n"; }
};

void manualManagement() {
    Resource* r = new Resource();
    // ... complex logic with early returns, exceptions ...
    delete r;  // ❌ May be skipped due to early return/exception
}

void raiiManagement() {
    std::unique_ptr<Resource> r = std::make_unique<Resource>();
    // ... complex logic ...
    // ✅ Destructor automatically called at scope exit
}
```

Memory Management in C++

Code Examples

📌 5. Exception Safety with RAII

Stack unwinding during exception propagation invokes destructors for all stack-allocated objects, including containers and smart pointers. This makes RAII-based code automatically exception-safe.

```
void exceptionUnsafe() {
    int* data = new int[1000];
    if (errorCondition()) {
        throw std::runtime_error("Error");  // ❌ data is leaked
    }
    delete[] data;
}

void exceptionSafe() {
    std::vector<int> data(1000);
    if (errorCondition()) {
        throw std::runtime_error("Error");  // ✅ vector destructor runs
    }
    // Automatic cleanup regardless of path
}
```

Memory Management in C++

Code Examples

📌 6. Preventing Double Delete

The C++ standard defines `delete nullptr` as a safe no-op operation. Setting pointers to nullptr after deletion provides defensive protection. However, unique_ptr provides compile-time prevention by disallowing copies.

```
void doubleDeleteBug() {
    int* p = new int(42);
    int* q = p;

    delete p;
    delete q;  // ❌ Double delete: UB
}

void preventDoubleDelete() {
    int* p = new int(42);
    delete p;
    p = nullptr;  // ✅ Safe to delete again (no-op)
    delete p;     // Safe: delete nullptr is defined as no-op
}

void modernApproach() {
    std::unique_ptr<int> p = std::make_unique<int>(42);
    // Cannot accidentally double-delete
    // Cannot copy (only move), enforcing single ownership
}
```

Memory Management in C++

Code Examples

📌 7. malloc/free vs new/delete Differences

The `new` operator performs two operations: memory allocation followed by constructor invocation. The `delete` operator invokes the destructor before deallocating memory. In contrast, `malloc` and `free` only manage memory without object lifecycle awareness.

```
#include <cstdlib>

class Widget {
    int value;
public:
    Widget(int v) : value(v) {
        std::cout << "Constructor: " << value << "\n";
    }
    ~Widget() {
        std::cout << "Destructor: " << value << "\n";
    }
};

void mallocStyle() {
    Widget* w = (Widget*)malloc(sizeof(Widget));
    // ❌ Constructor never called! Object in invalid state
    free(w);
    // ❌ Destructor never called! Resources leaked
}

void newStyle() {
    Widget* w = new Widget(42);
    // ✅ Constructor called, object properly initialized
    delete w;
    // ✅ Destructor called, resources cleaned up
}
```

Memory Management in C++

Code Examples

📌 8. Smart Pointer Ownership Semantics

Unique_ptr enforces exclusive ownership through move semantics and deleted copy operations. Shared_ptr implements reference counting, maintaining a count of how many shared_ptrs reference the same object, deleting it only when the count reaches zero.

```
#include <memory>

void uniqueOwnership() {
    std::unique_ptr<int> p1 = std::make_unique<int>(100);
    // std::unique_ptr<int> p2 = p1;  // ❌ Copy deleted
    std::unique_ptr<int> p2 = std::move(p1);  // ✅ Ownership transferred
    // p1 is now nullptr, p2 owns the resource
}

void sharedOwnership() {
    std::shared_ptr<int> p1 = std::make_shared<int>(100);
    std::shared_ptr<int> p2 = p1;  // ✅ Reference count = 2
    std::shared_ptr<int> p3 = p1;  // ✅ Reference count = 3
    // Resource deleted when all shared_ptrs destroyed
}
```

What You Learned

Key Takeaways

- ✓ Automatic Lifetime Management
- ✓ Manual Lifetime Management
- ✓ Direct Comparison
- ✓ Resource Acquisition Is Initialization

→ Swipe to learn →